MODULE 3
EXPERIMENT REPORT

Python BDD RESTful Automations

Student Name	**Srishti Chatterjee**
Enrollment Number	12023002022033
Department	CST
Institution	Institute of Engineering and Management, Newtown

1. Module and Experiment Title

Module 3.0: Python BDD RESTful Automations

Experiment: End-to-End REST API Automation with Python Requests, MySQL, Behave BDD, Hooks, Tags, and Allure Reporting.

2. Problem Statement

Manual API verification becomes increasingly repetitive when a project contains many endpoints, different request bodies, authentication mechanisms, response checks, database records, and execution environments. The purpose of this experiment is to build a reusable Python-based REST API automation framework that can exercise HTTP methods, inspect responses, manage authentication and sessions, use database-backed test data, and express API checks through Behaviour-Driven Development (BDD) scenarios.

3. Objectives

- Describe the main characteristics of REST services and distinguish them from SOAP-based services.
- Configure Python, the requests package, and Behave for automated API testing.
- Automate the major HTTP operations: GET, POST, PUT, PATCH, and DELETE.
- Check status codes, headers, JSON responses, text and binary payloads, and error conditions.
- Use reusable configuration, request data, authentication, sessions, and cookies.
- Connect Python API tests with MySQL to support data-driven execution.
- Create Behave feature files, step definitions, hooks, tags, and scenario outlines.
- Produce Allure-based execution reports for BDD API tests.

4. Tools, Software, and Concepts

- Python 3.x
- requests library
- Behave BDD framework
- Allure reporting framework
- MySQL with a Python database connector
- JSON, HTTP methods, headers, status codes, authentication, cookies, and sessions
- Configuration files/objects and reusable test data
- VS Code, PyCharm, Command Prompt, or a comparable development environment

5. Implementation and Source Code

5.1 Python Fundamentals, REST, and SOAP

REST is an architectural approach in which resources are accessed through standard HTTP operations. REST APIs commonly use JSON and are generally stateless. SOAP, in contrast, is a protocol that commonly uses XML envelopes and formal messaging conventions. This experiment uses Python together with requests for REST automation.

```
import requests

base_url = "https://example.com/api"
response = requests.get(f"{base_url}/users/1")
print(response.status_code)
print(response.json())
```

5.2 Python and Behave Environment Setup

```
python -m venv venv
venv\Scripts\activate
pip install requests behave allure-behave mysql-connector-python
```

A representative project layout is:

```
api_automation/
├── features/
│   ├── environment.py
│   ├── users.feature
│   └── steps/
│       └── user_steps.py
├── utilities/
│   ├── config.py
│   └── api_client.py
├── test_data/
│   └── payloads.json
└── requirements.txt
```

5.3 GET Request and JSON Validation

```
import requests

response = requests.get("https://example.com/api/users/1")
assert response.status_code == 200
data = response.json()
print(data)
```

The response object exposes useful information such as the HTTP status, headers, text, binary content, decoded JSON, cookies, and elapsed time. These properties can be used to build precise API assertions.

5.4 Status-Code and Header Checks

```
assert response.status_code == 200
assert "content-type" in response.headers
assert "application/json" in response.headers["content-type"]
```

5.5 POST Request with Payload and Headers

```
payload = {
    "name": "Ahana",
    "role": "QA Engineer"
}

headers = {"Content-Type": "application/json"}

response = requests.post(
    "https://example.com/api/users",
    json=payload,
    headers=headers
)

assert response.status_code in [200, 201]
```

5.6 Reusable Configuration, Authentication, Sessions, and Cookies

```
import requests

session = requests.Session()
session.headers.update({"Accept": "application/json"})
session.headers.update({"Authorization": "Bearer <token>"})

response = session.get("https://example.com/api/profile")
print(response.cookies)
```

Common configuration can be maintained in a configuration object or a dedicated file. Payloads may come from JSON, CSV files, or database records so that test logic remains reusable. Using requests.Session() also keeps shared headers, cookies, and connection state available across related requests.

5.7 MySQL-Driven API Testing

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="password",
    database="api_test"
)

cursor = connection.cursor(dictionary=True)
cursor.execute("SELECT id, name, role FROM users")
rows = cursor.fetchall()

for row in rows:
    print(row)

cursor.close()
connection.close()
```

The database can supply request inputs as well as expected values for assertions. Connections and cursors should be closed reliably after execution, preferably with structured exception handling or context-management techniques.

5.8 PUT, PATCH, and DELETE

```
put_response = requests.put(url, json=complete_payload)
patch_response = requests.patch(url, json={"role": "Tester"})
delete_response = requests.delete(url)

assert put_response.status_code in [200, 204]
assert patch_response.status_code in [200, 204]
assert delete_response.status_code in [200, 202, 204]
```

PUT is normally used to replace a complete resource representation, while PATCH is intended for partial changes. DELETE requests that a resource be removed. The exact successful status code is determined by the API contract.

5.9 Advanced Response Processing and Exception Handling

```
try:
    response = requests.get(url, timeout=10)
    response.raise_for_status()
    text_data = response.text
    binary_data = response.content
except requests.exceptions.Timeout:
    print("Request timed out")
except requests.exceptions.RequestException as error:
    print("Request failed:", error)
```

For large responses, streaming can be used with stream=True. Text can be obtained through response.text, binary data through response.content, and structured JSON through response.json(). Exception handling should differentiate HTTP failures, timeouts, connection problems, and invalid JSON responses.

5.10 Behave Feature Scenario

```
Feature: User API validation

@smoke @api
Scenario: Retrieve a user successfully
    Given the API endpoint is available
    When I send a GET request for user "1"
    Then the response status code should be 200
    And the response should contain the user id
```

5.11 Step Definitions

```
from behave import given, when, then
import requests

@given("the API endpoint is available")
def step_endpoint(context):
    context.base_url = "https://example.com/api"

@when('I send a GET request for user "{user_id}"')
def step_get_user(context, user_id):
    context.response = requests.get(
        f"{context.base_url}/users/{user_id}"
    )
```

```
@then("the response status code should be 200")
def step_status(context):
    assert context.response.status_code == 200
```

5.12 Hooks, Scenario Outlines, and Tags

```
# features/environment.py
def before_scenario(context, scenario):
    context.session = requests.Session()

def after_scenario(context, scenario):
    context.session.close()

# Selective execution
behave --tags=smoke
behave --tags=api

# Generate Allure results
behave -f allure_behave.formatter:AllureFormatter -o allure-results
```

Scenario outlines allow the same BDD flow to run with multiple data sets through Examples tables. Tags such as @smoke, @regression, and @api make it possible to select particular groups of scenarios.

5.13 Allure Reporting

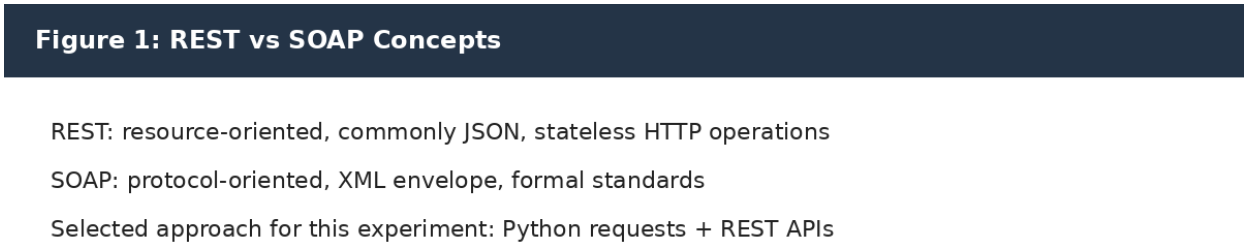
```
allure serve allure-results
```

Allure presents a visual execution summary containing scenarios, individual steps, statuses, failures, and other run details. The report is generated after the BDD test execution has produced its result files.

6. Evidence: Original Screenshots

The following evidence figures are reproduced directly from the supplied PDF without redesigning or altering their visual content. They document the expected execution or framework state for the corresponding stages of the experiment.

Figure 1: REST vs SOAP Concepts



Observation: The figure presents the REST and SOAP comparison used to introduce the selected Python requests + REST approach.

Figure 2: GET Request and JSON Response

Figure 2: GET Request and JSON Response

```
GET /users/1
HTTP 200 OK
JSON response: {"id": 1, "name": "Example User"}
Response converted using response.json()
```

Observation: The figure shows a GET operation, an HTTP 200 response, a JSON payload, and conversion through response.json().

Figure 3: Response Validation

Figure 3: Response Validation

```
Status code assertion: PASSED
Content-Type header: application/json
Required JSON key validation: PASSED
Response time and error checks recorded
```

Observation: The figure records status-code, Content-Type, JSON validation, response-time, and error checks.

Figure 4: POST Request with Payload and Headers

Figure 4: POST Request with Payload and Headers

POST /users

Headers: Content-Type: application/json

Payload: {"name":"Ahana","role":"QA Engineer"}

HTTP 201 Created

Returned resource identifier captured

Observation: The figure shows a POST request with JSON headers and payload, including a 201 Created response.

Figure 5: Configuration, Authentication and Session

Figure 5: Configuration, Authentication and Session

Base URL loaded from config object

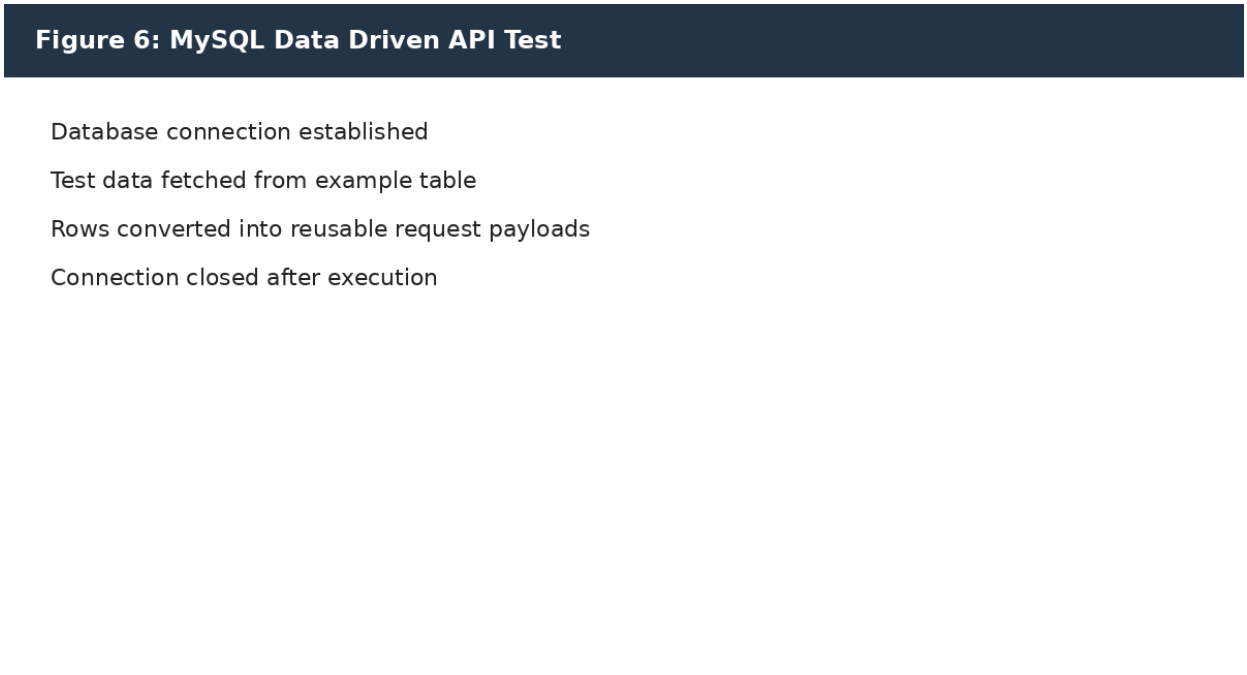
Bearer token passed through auth/header

requests.Session() reused connection and common headers

Cookies managed at session level

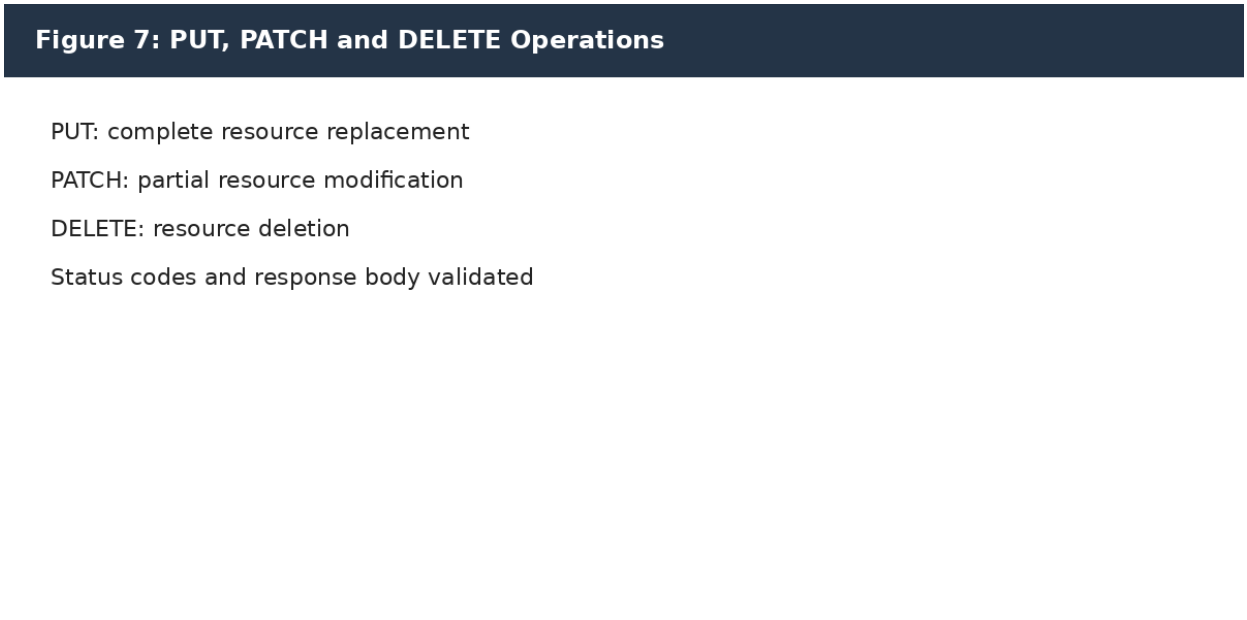
Observation: The figure summarizes configuration loading, bearer-token handling, session reuse, and cookie management.

Figure 6: MySQL Data Driven API Test



Observation: The figure shows database connectivity, retrieval of test data, reusable payload preparation, and connection closure.

Figure 7: PUT, PATCH and DELETE Operations



Observation: The figure distinguishes complete replacement, partial modification, and resource deletion while noting validation of responses.

Figure 8: Advanced HTTP Response Handling

Figure 8: Advanced HTTP Response Handling

Text content read with response.text

Binary content read with response.content

Streaming handled with stream=True

Exceptions handled using requests exceptions

Observation: The figure covers text and binary content, streaming, and exception handling.

Figure 9: Behave Step Definitions and Hooks

Figure 9: Behave Step Definitions and Hooks

Feature file uses Given / When / Then

environment.py contains before_scenario and after_scenario

Step definitions call reusable API client functions

Scenario context stores response data

Observation: The figure illustrates Given/When/Then usage, scenario hooks, reusable API functions, and scenario context.

Figure 10: BDD Tags and Allure Reporting

Figure 10: BDD Tags and Allure Reporting

Tags: @smoke @regression @api

Selected scenarios executed by tag

Allure results generated from test run

Report displays passed, failed and skipped scenarios

Observation: The figure summarizes tagged execution and generation of Allure results and report views.

7. Result, Observation, and Conclusion

The experiment establishes a Python-based approach for automating REST API testing across the main stages of the testing lifecycle. GET, POST, PUT, PATCH, and DELETE operations can be executed and checked through the requests library. Reusable configuration, payloads, authentication, sessions, cookies, and MySQL-backed data improve the structure and maintainability of the framework.

Behave introduces a readable BDD layer through feature files, step definitions, hooks, scenario outlines, and tags. Allure adds execution visibility through generated reports. Overall, the implementation demonstrates how modular, reusable, and data-driven components can be combined for REST API automation.